AI Engineer Take-Home: Multi-Tenant Agent Platform

Oct 30, 2026 · @H

Overview

Build one chat platform that hosts AI agents for two different client organisations. Each organisation has its own agent, knowledge base, customers and data. A customer of one organisation must never be able to reach the other organisation's agent or data.

- **Agent A** works for an online store: it answers product questions and takes, edits, tracks and repeats orders.
- **Agent B** works for a service business: it answers service questions and books, reschedules and cancels appointments without conflicts.
- **Stretch:** an admin agent that lets each organisation's staff ask about their own orders, bookings and conversion.

Deadline: 4 days from receiving this brief. A fifth day is allowed as a grace period for final submission, but submissions made within 4 days are favoured. Get the core working end to end before touching stretch goals — a solid core beats a half-built everything.

This task tests agent engineering, multi-tenant security, business-rule correctness, retrieval, evaluation and product sense.

Tenant isolation and auth

A user who signs in to Org A can use only Org A's agent, and sees only their own orders, bookings and chats. We will actively try to break this.

- [ ] Each user belongs to exactly one organisation and has a role: `customer` or `admin`.
- [ ] Sign-up happens on an organisation's own entry point (e.g. `/a/himali-tea` or `himali-tea.yourapp.local`), which fixes the user's `org_id`.
- [ ] Sign up, log in, log out with hashed passwords (bcrypt or argon2) or OAuth. A managed provider (Clerk, Auth0, Supabase Auth) is fine — explain the choice.
- [ ] Secure session handling — explain your choice.

- [ ] An Org A user can't reach Org B's agent or data by any route — including by changing the URL, headers or request body.
- [ ] Knowledge-base search never returns another organisation's content, whatever the user or the model asks for.
- [ ] Rate limit the chat endpoint (e.g. 20 messages/minute per user). No secrets in the repo; provide `.env.example`.
- [ ] Automated tests proving cross-org and cross-user access fails.

The two organisations and their knowledge bases

You create the content for two fictional organisations, seed it, and load the unstructured parts into a vector database. Write the content yourself or generate it with an LLM — just keep it consistent.

	Org A: Himali Tea House	Org B: Lotus Wellness Studio
Business	Online store selling tea and teaware	Massage, physiotherapy and yoga studio
Knowledge base (vector DB)	About us, shipping, returns and refund policy, FAQ (at least 15 Q&As), product descriptions and brewing guides	About us, service descriptions, practitioner bios, cancellation and late policy, FAQ (at least 15 Q&As)
Structured data (relational DB)	At least 20 products: SKU, name, category, price, stock	At least 6 services (name, duration, price), 3 practitioners, weekly opening hours, holidays
Seed customers	3 customers, 2 with past orders	3 customers, 2 with past bookings

Rules for the knowledge base

- Answers about prices, stock, order status and availability must always match the live data.
- Answers from the knowledge base must cite the source document and section.
- Provide a single command that rebuilds both knowledge bases from the seed files (e.g. `make seed`).
- Explain your chunking and embedding choices in the README.

Agent A: ordering agent for Himali Tea House

The agent answers questions about the store and its products, and handles the full order life cycle in chat. Orders move through `pending → confirmed → shipped → delivered`; only a `pending` order can be changed or cancelled.

Tools (minimum)

Tool	What it does
`search_knowledge`	Searches Org A's knowledge base for policies, FAQs and product details
`search_products`	Finds products by name, category or price range, with live price and stock
`create_order`	Places an order for the signed-in customer from a list of SKUs and quantities
`update_order`	Changes items, quantities or delivery address — only while the order is `pending`
`cancel_order`	Cancels an order — only while it is `pending`
`get_order_status`	Tracks one order: status, items, total and status history
`list_my_orders`	Lists the customer's past orders, newest first
`reorder`	Creates a new `pending` order from a previous one, re-checking stock and current prices

Business rules

1. Before placing or changing an order, the agent shows a summary — items, quantities, current prices, total — and waits for the customer to confirm.
2. Quantity can't exceed stock. If an item is short, the agent says so and offers an alternative.
3. An edit or cancel on a non-`pending` order is refused with the reason and the current status.
4. A customer can only see and act on their own orders.
5. "Same as last time" works: the agent finds the previous order and proposes it, noting any price or stock changes.

6. Payment can be mocked (e.g. cash on delivery). Provide an admin endpoint or script that advances order status, so tracking and the edit rules can be demonstrated.

Agent B: booking agent for Lotus Wellness Studio

The agent answers questions about services, prices, practitioners and opening hours, and books, reschedules and cancels appointments. The studio's time zone is Asia/Kathmandu (UTC+05:45); all times shown to customers use it.

Tools (minimum)

Tool	What it does
`search_knowledge`	Searches Org B's knowledge base for services, policies and FAQs
`list_services`	Returns services with duration and price
`get_opening_hours`	Returns weekly hours and upcoming holidays
`find_available_slots`	Returns free slots for a service, date range and optional practitioner
`create_booking`	Books a slot for the signed-in customer
`reschedule_booking`	Moves a booking to a new slot — only if the new slot is free
`cancel_booking`	Cancels a booking, applying the cancellation policy
`list_my_bookings`	Lists the customer's upcoming and past bookings

Business rules

1. A slot must sit inside opening hours, avoid holidays, and fit the service's full duration.
2. No overlaps: a practitioner can't be double-booked, and a customer can't hold two bookings at the same time.
3. Booking and rescheduling must be safe when two customers grab the same slot at once; explain how you guarantee this.
4. A reschedule that would conflict is refused, and the agent offers the nearest free alternatives.

5. The agent confirms service, practitioner, date, time and price before booking or rescheduling.
6. Relative dates ("next Tuesday afternoon") resolve correctly in the studio's time zone.

Scheduling: build or integrate

You may write your own availability engine, or integrate a scheduling API. Cal.com API v2 has endpoints for free slots, creating, rescheduling and cancelling bookings. Google Calendar's freeBusy query returns busy intervals you can check conflicts against. If you integrate one, keep it behind an interface so the agent's tools don't depend on the vendor, and say how you'd handle the API being down.

Chat interface

- [ ] Each organisation's entry point shows its own name, colours and a greeting from its agent; after login the user lands straight in their organisation's chat.
- [ ] Sidebar with the user's conversations: new chat, rename, delete.
- [ ] Message thread with markdown and code blocks; responses stream token by token, with a **Stop** button.
- [ ] Visible tool activity (e.g. "Checking stock…", "Finding free slots…") and clickable citation chips for knowledge-base answers.
- [ ] Rich cards where they help: an order summary with a **Confirm** button, a list of time slots to pick from, an order or booking status card.
- [ ] A "My orders" or "My bookings" panel that matches what the agent says.
- [ ] Enter sends, Shift+Enter adds a line; empty, loading and error states; usable at 375 px wide.

Stretch goals

Admin analytics agent (most valued stretch)

Users with the `admin` role get a separate chat with an analytics agent for their own organisation. It answers questions such as "How did we do today?", "How many new clients this week?" or "Which practitioner is busiest?".

Metric	Org A (store)	Org B (studio)
Activity summary	Orders by status, revenue, average order value	Bookings by status, cancellations, practitioner utilisation
Clients today	Unique customers who chatted; new sign-ups	Unique customers who chatted; new sign-ups
Conversion rate	Conversations that led to an order ÷ all conversations in the period	Conversations that led to a booking ÷ all conversations in the period
Top items	Best-selling products; items most often out of stock	Most-booked services; busiest time slots

Requirements if you build it:

- Read-only: the admin agent cannot create, change or delete anything.
- Scoped to the admin's own `org_id`; a customer account gets `403`.
- Explain how you enforce both.
- State the time window and definition behind every number it reports.

Other extras

- Add a third organisation through configuration and seed data alone, with no code changes
- Long-term customer memory (e.g. "my usual tea", preferred practitioner)
- Booking reminders or order-shipped notifications by email
- Tracing of each agent run (Langfuse, LangSmith or OpenTelemetry) and per-org token cost
- LLM-as-judge scoring in the eval suite
- One-command start with Docker Compose, or a deployed URL with test accounts

Evaluation (required)

Ship an eval suite of at least **12 scenarios per agent** that runs with one command (e.g. `make eval`), reports pass/fail per scenario, and checks the database state afterwards — not just the reply text. Design the scenarios yourself. Cover knowledge answers, questions the knowledge base can't answer, the happy path, every business rule, memory, edge cases, cross-organisation access and prompt injection.

In the README, report the results and explain each failure and how you'd fix it.

Tech guidance

Use any stack you're productive in. If you want a default:

Layer	Suggested	Also fine
Frontend	Next.js + TypeScript + Tailwind	React + Vite, SvelteKit
Backend	FastAPI (Python)	Node/Express, NestJS
Relational DB	Postgres	SQLite for local dev
Vector DB	pgvector	Qdrant, Chroma, Weaviate
LLM (free)	Gemini API free tier, or Mistral API free Experiment plan	A local model via Ollama
Embeddings (free)	Gemini embedding model, or `mistral-embed`	A local embedding model
Scheduling	Your own slot engine	Cal.com API v2, Google Calendar API
Streaming	Server-Sent Events	WebSockets

Rules

- Agent frameworks (LangGraph, LlamaIndex, Vercel AI SDK, OpenAI Agents SDK) are allowed, but you must be able to explain the agent loop line by line.
- Don't use a hosted end-to-end "chat with your files" product. We want to see your retrieval.
- AI coding assistants are allowed. Say in the README how you used them.
- Use a free LLM tier: Gemini API (API key from Google AI Studio) or Mistral's Experiment plan (no card; phone verification). Make the provider switchable with an environment variable. Free tiers have tight rate limits, so plan for them. Use only the fictional seed data — Mistral's free plan may use requests to train its models.

Deliverables and submission

Reply to the brief email within 4 days (or by the end of the fifth, grace day at the latest) with links to:

1. **Git repository** (private is fine — share it with the reviewer we name)
2. **README** covering:
   - Setup in five commands or fewer, including `make seed`
   - Test logins: one customer and one admin per organisation
   - Architecture overview and how tenant isolation is enforced
   - Key decisions: chunking, model, scheduling build vs integrate, concurrency handling
   - Eval results and failure analysis
   - What you'd do with another week, and how you used AI coding tools
3. **`.env.example`** listing every variable needed
4. **Demo video, 5–8 minutes** showing: an Org A order placed, edited, tracked and reordered; an Org B booking made and a conflicting reschedule refused; an Org A user failing to reach Org B; and the admin agent if built
5. **Optional:** a deployed URL

How we evaluate

We score these areas, weighted as shown.

Area	Weight
Agent engineering	20%
Tenant isolation and auth	20%
Business rules	15%
Evaluation	15%
Knowledge base and retrieval	10%
Chat UX	10%
Code and communication	10%
Admin agent (stretch)	bonus